

PERFECT LABELS FOR FREE? EXACT LABELS TEACH SHORTCUTS, EXACT TRACES TEACH SHORT PROGRAMS

Anonymous authors

Paper under double-blind review

ABSTRACT

Exact verifiers (primality tests, SAT solvers, unit tests) label data with no noise at no marginal cost, and they supply the rewards of verifiable-reward post-training. We use them as a measurement instrument: with label noise removed, the effect of the supervision *format* can be isolated. We fine-tune Qwen2.5-1.5B-Instruct with LoRA on 180 exactly labeled examples per task and compare supervision by the answer alone with supervision by a deterministic procedure trace followed by the answer, against a length-matched scrambled-trace control. Answer-only training recovers about what simple probes on the input’s surface recover. On 4-digit primality it reaches 82.3%, between a one-hot linear probe (79.3%) and the best cross-validated probe (89.7%); its answers coincide with the rule “the last digit is 1, 3, 7 or 9” on 99.6–100.0% of test items, and it misclassifies all 15 odd composites without a factor ≤ 7 in the test set. On 4-digit divisibility by 7, which has no surface rule, answers stay at chance (50.3%) while a 4-step long-division trace reaches 88.3% and its scrambled counterpart 50.0%. On primality, whose trial-division trace runs up to 25 steps, traces hurt (57.7%). The results fit a simple map: answer supervision works when the task is surface-learnable, trace supervision works when the per-step accuracy compounded over the trace length stays high. We preregister tests of the map (a divisor family, a 6-digit length test, hard negatives, an answer-then-trace arm, prompting controls, a larger model) and report every cell. The lesson concerns how to design supervision for verifiable-reward pipelines, not how to do arithmetic.

1 INTRODUCTION

Verifiable-reward post-training rests on programs that decide correctness exactly: answer checkers for mathematics, unit tests for code, solvers for logic (Lambert et al., 2024; DeepSeek-AI et al., 2025). Such verifiers are often described as a source of perfect labels for free. They are perfect in one sense: a verified label is never wrong. Whether a model trained on those labels learns the computation the verifier performs, or a cheaper function that agrees with it on the training distribution, is a separate question, and in practice it is confounded with label noise, with task difficulty, and with the format of the supervision.

We use exact verifiers to remove the first confound. When every label is correct, any difference between two training regimes on the same inputs is a difference in what the supervision exposes. We choose small decision tasks whose structure is known in advance (which surface rules approximate the label, how many steps the natural procedure takes) so that what should be learnable can be stated before training. The point is not to teach a language model what a ten-line program already computes. It is that on such tasks we can tell which function the model learned, which is what a verifiable-reward pipeline for mathematics or code needs to know about its own supervision and usually cannot measure.

We compare two supervision formats on Qwen2.5-1.5B-Instruct. *Answer-only* supervision (arm A) teaches a shortcut. On 4-digit primality the answer-trained model reaches 82.3%, between a one-hot linear probe on the digits (79.3%) and the best cross-validated surface probe (89.7%). Its answers are a one-token rule: they coincide with “the last digit is 1, 3, 7 or 9” on 99.6–100.0% of test items,

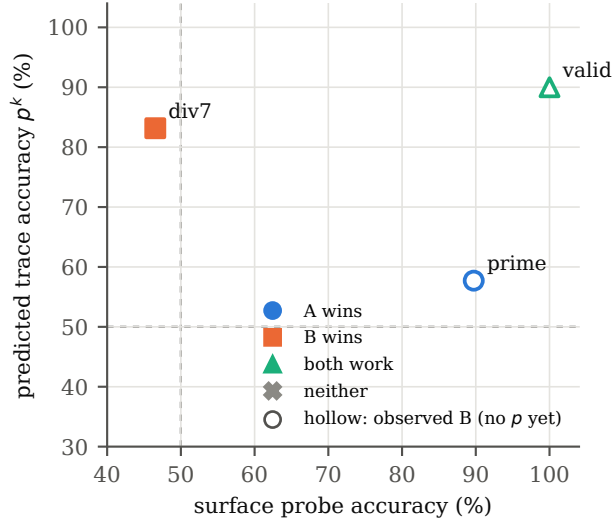


Figure 1: The map. Each task is placed by the accuracy of the surface probe trained on answer labels (horizontal) and by the predicted trace accuracy p^k (vertical), where p is the per-step accuracy of arm-B generations and k the number of steps (hollow markers: observed B accuracy, for tasks whose generations were not saved); color and shape mark the winning arm (A, B, both, neither). The map predicts A to win on the right, B at the top, both at top right and neither at bottom left.

and that rule alone scores 82.5%. On divisibility by 7, where no surface rule exists, it stays at chance. *Trace* supervision (arm B: a deterministic procedure trace, then the answer) succeeds when the trace is short. The 4-step long-division trace lifts divisibility by 7 from 50.3% to 88.3% (three seeds, McNemar $p \leq 7 \times 10^{-13}$ in each), while a scrambled trace with the same tokens (arm C) stays at 50.0%. On primality, where the trial-division trace of a prime has 11–25 steps, traces hurt (57.7% vs. 82.3%), and the loss sits where the trace is long: 19.6% of primes are classified correctly against 95.8% of composites, whose trace usually stops after 1 step.

These observations fit a two-condition map (Figure 1). Answer supervision works if the label is *surface-learnable*, which a probe measures before any fine-tuning. Trace supervision works if the trace is *executable*: with per-step accuracy p and k steps, the chance of an error-free trace, about p^k , must stay high. The map makes predictions we had not tested when we wrote it down, and we preregistered them (§3): varying the divisor at fixed k , lengthening the input to $k = 6$, hard negatives for the primality shortcut, an answer-then-trace arm that separates the trace as training signal from the trace as test-time computation, prompting controls on the base model, and a larger model.

Contributions.

- A controlled comparison of supervision formats under exact labels (answers, procedure traces, length-matched scrambled traces, shuffled labels) with paired tests on problem-disjoint test sets (§3).
- Evidence that exact answer labels teach the surface probe’s function rather than the verifier’s: probe-level accuracy, rule agreement and failure on hard negatives (§4.1).
- Evidence that exact traces teach short programs: the trace unlocks divisibility by 7 through its content, not its tokens, and fails on primality with the per-class signature of errors compounding over trace length (§4.2).
- A predictive map with preregistered tests whose results are all reported (Table 3), and a remark relating it to automata and to known results on scratchpads (§6).

One result reversed an earlier conclusion of ours. A verdict written on 2026-08-04 from a 60-example trace run stated that traces do not unlock divisibility by 7; the preregistered 180-example cell, which finished afterwards with the same code, gave 92.5%. We report both. Before any new

run started, the replication of that cell was made a go/no-go condition for this submission, and it passed: 90.0% and 82.5% in seeds 1 and 2 (§4.2).

2 RELATED WORK

Intermediate steps as supervision. Scratchpads (Nye et al., 2021) and chain-of-thought (CoT) prompting (Wei et al., 2022) showed that emitting intermediate steps improves multi-step computation, and Sprague et al. (2025) find the benefit of CoT concentrated on mathematical and symbolic tasks. For small transformers trained from scratch, Lee et al. (2024) show that the format of arithmetic data, including step-by-step formats, changes sample efficiency sharply, and Charton (2024) explains a transformer’s GCD predictions by rules built from divisors of the number base, the kind of surface rule we probe for. Dziri et al. (2023) show how errors compound across the steps of compositional tasks. We fine-tune a pretrained model, vary only the completion, and hold labels exact.

Theory. Parity-like targets give gradient-based learners almost no signal from end labels (Shalev-Shwartz et al., 2017). Sub-task decomposition makes otherwise unlearnable sequence-to-sequence tasks learnable (Wies et al., 2023); CoT raises the expressivity of bounded-depth transformers (Feng et al., 2023; Li et al., 2024); Abbe et al. (2024) define the globality degree of a target, show that it bounds what transformers learn efficiently, and show that scratchpads, in particular an inductive scratchpad, can break that barrier; Kim & Suzuki (2025) prove that a transformer trained with teacher-forced intermediate parities learns parity efficiently. Section 6 reads divisibility through this lens without extending these results.

What in a trace matters. Valmeekam et al. (2026) train transformers from scratch on formally verifiable traces and find the semantics of the trace less important than usually assumed: correct-trace training yields invalid traces with correct answers, and unrelated traces can still help. Our scrambled control is a coarser intervention (token multiset kept, order destroyed); on divisibility it removes the entire gain. Process supervision rewards steps rather than outcomes (Lightman et al., 2024), and program-generated logic corpora improve LLM reasoning (Morishita et al., 2024).

Verifiers in post-training. STaR fine-tunes on self-generated rationales that reach verified answers (Zelikman et al., 2022); reinforcement learning with verifiable rewards (RLVR) drives recent reasoning models (Lambert et al., 2024; DeepSeek-AI et al., 2025). Fine-tuning on model-generated correct solutions can amplify spurious correlations, which per-step credit from negatives mitigates (Setlur et al., 2024); SFT and RL generalize differently (Chu et al., 2025); and RLVR mostly reweights what the base model already samples (Yue et al., 2025). Our answer arm is an instance of shortcut learning (Geirhos et al., 2020); exact labels do not prevent it.

3 SETUP

Exact verifiers as an instrument. Each task has an exact decision procedure: trial division for primality, the remainder for divisibility, and Z3 (de Moura & Bjørner, 2008) for validity (premises entail C iff premises $\wedge \neg C$ is unsatisfiable). Labels are correct by construction, every arm sees the same prompts and labels, and arms differ only in the completion the model is trained to produce. The tasks are chosen so that the cheap functions that approximate each label are known.

Tasks. Table 1 summarizes them. **prime:** is a uniformly drawn 4-digit n prime? The trace performs trial division by the primes up to $\lfloor \sqrt{n} \rfloor$, one “ $n \bmod p = m$.” step per prime, and stops at the first divisor, so a prime needs 11–25 steps (mean 19.2) and a composite a median of 1. **div7:** is a 4-digit n divisible by 7? The trace performs long division, $r \leftarrow (10r + x_t) \bmod 7$ for each digit x_t , so $k = 4$ for every item. **valid:** does a propositional conclusion follow from its premises? Arguments instantiate 12 forms (6 valid, 6 fallacies) with random atom letters; the trace is the Z3 certificate, a countermodel or an unsatisfiable core. Train and test share forms but not instances, so this task is surface-learnable by design and serves as a check (Appendix B). Each test set has 240 items (120 per class), is problem-disjoint from training, and is identical across arms; training sets are balanced samples of $n \in \{60, 180, 540\}$ from a training pool.

Table 1: Tasks. k : steps in the trace (prime: for primes; a composite’s trace stops at its smallest prime factor, after a median of 1 step). Tokens: mean whitespace tokens per trace. Probe: the surface probe selected by 5-fold cross-validation on the arm-A training items (§4.1), test accuracy (%) at $n = 180$.

task	trace procedure	k	tokens	probe
prime	trial division by the primes $\leq \lfloor \sqrt{n} \rfloor$	11–25	71.5	89.7
div7	long division, $r \leftarrow (10r + x_t) \bmod 7$	4	58.0	46.5
valid	Z3 countermodel or unsatisfiable core	certificate	37.0	100.0

Table 2: Accuracy (%) at $n = 180$, Qwen2.5-1.5B-Instruct, 240 test items per task (Wilson half-width about 6 pp). A: answers; B: trace, then answer; C: scrambled trace. A, B, C: mean over seeds (div7 A and B: 3 seeds, including the preregistered replication S1; otherwise two). p : exact McNemar, per seed. Per-seed values in Appendix A.

task	base	probe	A	B	C	B vs. A, p	B vs. C, p
prime	59.6	89.7	82.3	57.7	52.1	$6 \times 10^{-7}, 2 \times 10^{-7}$	0.22, 0.33
div7	62.5	46.5	50.3	88.3	50.0	$4 \times 10^{-27}, 2 \times 10^{-18}, 7 \times 10^{-13}$	$1 \times 10^{-25}, 1 \times 10^{-20}$
valid	30.0	100.0	99.6	90.0	52.5	$0.50, 7 \times 10^{-15}$	$2 \times 10^{-33}, 5 \times 10^{-10}$

Arms. **base**: the model zero-shot. **A** (answers): the completion is “Answer: Yes” or “Answer: No”. **B** (trace, then answer): the deterministic trace, a newline, then the answer (examples in Appendix C). **C** (scrambled trace): B’s traces are first permuted across training examples, so that trace length cannot signal the label (a composite’s or an invalid argument’s trace is shorter), then token-shuffled within each trace; the answer line keeps the example’s own true label. Over the training pool, C has exactly B’s multiset of whitespace tokens; in the sampled training sets the totals differ by at most 7%. **A-full**: answers only, on the whole training pool (1,320 examples for prime, 1,600 for valid). **Shuffled labels**: from an earlier study with the same model and LoRA configuration (240 training items, answer-only prompts), with labels permuted across training items. Preregistered additions: **B'** (answer, then trace) on div7, in which the answer is emitted before the trace and so cannot be computed by it; and, on the base model only, a zero-shot CoT prompt and 4-shot trace demonstrations.

Training and evaluation. LoRA (Hu et al., 2022) with $r = 8$, $\alpha = 16$, dropout 0.05 on the query, key, value and output projections of Qwen2.5-1.5B-Instruct (Qwen et al., 2024); AdamW, learning rate 2×10^{-4} , batch size 1, 3 epochs, bf16; loss on completion tokens only, prompts in the chat template. Evaluation is greedy generation (at most 512, 176 and 320 new tokens for prime, div7 and valid), and one extractor serves every arm: the last “Answer: Yes/No”, else the first standalone yes/no. Each run is a fresh process on one 8 GB consumer GPU and takes 1–17 minutes at $n = 180$.

Statistics, preregistration and provenance. Wilson 95% intervals at 240 test items are about ± 6 pp. Paired comparisons use the exact McNemar test (McNemar, 1947) on per-item correctness within a seed, with Holm correction (Holm, 1979) across tasks within each hypothesis family at $\alpha = 0.05$. Two preregistrations govern the paper. The first was frozen before the prior grid of 36 runs and fixed arms, tasks and four endpoints: B beats A at matched examples (H1), B beats C (H2), B is more efficient per training token (H3), and on div7 B reaches about 60% with A at chance (H4). The second (Appendix D) was committed before any run it lists and fixes the decision rules S1–S6 of Table 3; a “win” is a difference of at least 10 pp with both seeds agreeing in sign. The surface probe belongs to neither: we defined it after seeing the prior grid and apply the same definition to every new task. *Provenance.* The prior study’s written verdict (2026-08-04) stated that the trace does not unlock div7. It was written from the 60-example trace run alone (50.0%) while the preregistered 180-example cell was still running; that cell then finished at 92.5% with no code change. The second preregistration made its replication (S1) a go/no-go condition for this submission; it passed (§4.2).

4 RESULTS

4.1 EXACT ANSWER LABELS TEACH THE SURFACE

Answers track surface probes. We fit simple probes on the same 180 training items as arm A and score them on the test set: logistic regression, a one-hidden-layer MLP and k -nearest neighbours, on 40 one-hot digit-by-position features alone or together with engineered divisibility features (last digit even; last digit 0 or 5; digit sum mod 3 and mod 9; alternating digit sum mod 11). For arguments the features are word and character n -grams, with atoms renamed in order of first appearance. Hyperparameters and the reported probe are chosen by 5-fold cross-validation on the training items. On div2, div3 and div11 the engineered features contain the answer ($n \bmod 2, 3, 11$), so there the probe uses one-hot features only. A tracks the probes on all three tasks (Table 2): on prime it lies between the one-hot linear probe (79.3%) and the best probe (89.7%), on div7 both are at chance (50.3% vs. 46.5%), and on valid both are near ceiling (99.6% vs. 100.0%).

The primality shortcut. A’s answers (recovered from its per-item correctness) coincide with the rule “the last digit is 1, 3, 7 or 9” on 99.6% and 100.0% of test items in seeds 0 and 1, and that rule alone scores 82.5%, within a point of A’s accuracy. On this test set A has learned divisibility by 2 and 5, the two divisors visible in the last digit, and nothing else: it calls 99.6% of primes prime and gets only 65.0% of composites right. On the same test set, “odd” scores 76.2%, “no factor 2 or 3” 86.7% and “no factor ≤ 7 ” 93.8%; A sits between the first two, and a probe with engineered divisibility features does better than A (89.7%). On the earlier study’s independent test set, answer training gave 82.9%. The test set contains 15 odd composites with no prime factor ≤ 7 , and A misclassifies every one of them in both seeds (correct: 0 and 0). Such hard negatives are 12.5% of all 4-digit composites, so a rule that ignores them costs little on the training distribution. The preregistered test S6 evaluates the prime adapters on a dedicated set of 240 numbers, 120 hard negatives and as many primes, on which the rule calls every item prime: A reaches [TBD]% ([TBD]% on the hard negatives; B: [TBD]%; S6 [TBD]).

More answers do not buy the program. On prime, A reaches 59.6%, 82.3% and 75.0% at $n = 60, 180$ and 540 , and 80.8% with the full pool of 1,320 examples. On div7, A is at chance at every n (50.0%, 50.3% and 50.0% at $n = 60, 180$ and 540), and what it learns is a seed-dependent constant answer: seed 0 answers “no” to most items (25.8% correct on multiples of 7, 78.3% on non-multiples), seed 1 “yes” (82.5% and 19.2%). A is also below the base model on div7 (62.5%; McNemar $p = 0.047$ and 0.0046): the base model answers after long free-form responses (its evaluation on prime takes 38 times as long as A’s), and answer-only training removes them. Part of what a trace adds may therefore be test-time computation; B’ and the prompting controls (§4.3) separate the two.

The gain is label correctness. With labels permuted across training items, the earlier study’s answer arm sat at 48.5–52.5% on all seven of its tasks, against up to 99.8% with correct labels (Appendix A). Correct labels are what drive A; on primality, what correct labels buy is the rule.

4.2 EXACT TRACES TEACH SHORT PROGRAMS

Divisibility by 7. Over three seeds B reaches 88.3% (92.5, 90.0 and 82.5) against 50.3% for A (52.1, 50.8 and 47.9; McNemar $p = 4 \times 10^{-27}, 2 \times 10^{-18}$ and 7×10^{-13}), and C stays at 50.0% ($p \leq 1 \times 10^{-20}$ against B). Seeds 1 and 2 are the preregistered replication, which passed its criterion (S1 passed), and a re-run of seed 0 with the generation-saving worker reproduced 92.5%. The gain holds on both classes, 82.2% on multiples of 7 and 94.4% on non-multiples, so it is not a constant answer. At $n = 60$ the same arm was at 50.0% (one seed); the effect appears between 60 and 180 examples.

What the trace errors look like. In the saved generations (seeds with generations saved: 1), every trace has all 4 steps, digits and carried remainders are copied without error (100.0% and 100.0%), and every step error is a wrong lookup of $(10r+x) \bmod 7$. A step is right with probability $p = 0.955$ given that the previous one was, so $p^4 = 83.2\%$ of traces should be error-free, and 82.9% are. When a trace contains an error, the answer is still right 56.1% of the time, close to the one half that the

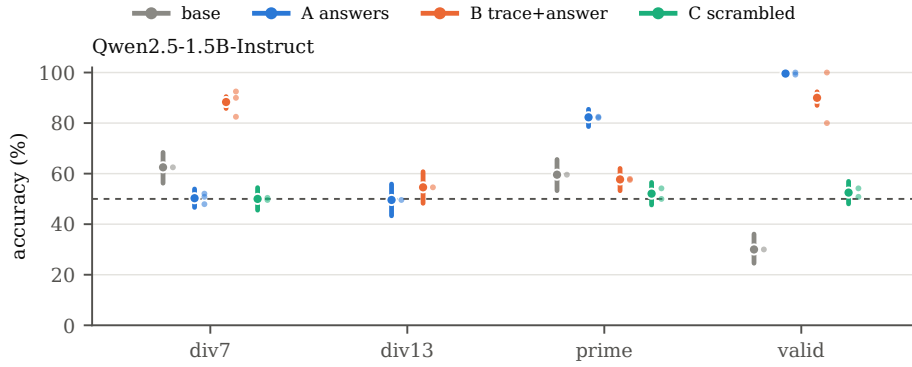


Figure 2: Accuracy by arm and task at $n = 180$, pooled over seeds with 95% Wilson intervals (240 test items per task and seed); small dots are single seeds, one row per model. Dashed line: majority class.

balanced-set approximation of §6 assumes; that approximation, $(1 + p^4)/2 = 91.6\%$, is close to the observed accuracy of 92.5%. None of this was preregistered.

Other divisors (S2, S3). Holding $k = 4$ and varying d , the map predicts that answers suffice for $d = 2$, whose label is the last digit, and that traces are needed for $d \in \{3, 11, 13\}$, which are coprime to 10 (§6). Table 3 gives the results (S2 [TBD]; S3 [TBD]). In the earlier answer-only study, $d = 3$ gave 67.3% and $d = 13$ 50.0%.

Primality: a long trace hurts. B reaches 57.7% against 82.3% for A ($p = 6 \times 10^{-7}$ and 2×10^{-7}), and B vs. C is not significant ($p = 0.22$ and 0.33): on primality the content of the trace gives no detectable benefit over its scrambled tokens. The per-class split locates the failure. A prime’s trace must get 11–25 consecutive “ $n \bmod p \neq 0$ ” steps right, and one false zero turns it into a composite; a composite’s trace usually ends at its first step. B classifies 19.6% of primes and 95.8% of composites correctly. More traces help the long class: at $n = 540$, B reaches 68.8% with 46.7% of primes correct, still below A.

Length (S5). On 6-digit div7 ($k = 6$) the preregistered prediction is $B \approx p^6 = 75.8\%$, with $p = 0.955$ measured on the 4-digit B generations. The primary cell applies the 4-digit B adapter to 6-digit inputs: [TBD]% (S5 [TBD]). Trained on 6-digit traces instead, B reaches [TBD]% (A: [TBD]%). The balanced-set form, $(1 + p^6)/2 = 87.9\%$, was not preregistered and is reported as a sensitivity analysis.

Validity. Both formats work: A 99.6%, B 90.0% (100.0% and 80.0% by seed; B is below A in seed 1, $p = 7 \times 10^{-15}$), C 52.5%. Because the 12 forms are shared between training and test, this task measures surface learnability, which the probe confirms (Appendix B).

Cost. Under the Qwen tokenizer, which splits numbers into single digits, B’s training completions contain 14–47 times as many tokens as A’s (33,575 vs. 720 per 180-example draw on prime). Where answers work, they are far cheaper, so the first preregistration’s efficiency hypothesis H3 failed on prime and valid; on div7, where answers never leave chance at any n we ran, it holds trivially. In sum, of the first preregistration’s endpoints, H1 ($B > A$) held on div7 only, H2 ($B > C$) held on div7 and valid but not on prime, H3 failed where answers work, and H4 held at $n = 180$ but not at $n = 60$.

4.3 CONTROLS

Content, not tokens. C sits at 50.0% on div7 and 52.5% on valid, far below B, although it trains on B’s tokens with the true answers. What helps on these tasks is the trace’s content. On prime, B and C are statistically indistinguishable, and both are below A.

Table 3: Preregistered tests (Appendix D) with results as they came out. Qwen2.5-1.5B-Instruct and 4-digit inputs unless stated; accuracy in %, mean of two seeds unless stated; McNemar per seed, Holm across tasks.

	cell	probe	A	B	criterion	met
S1	div7, seeds 1 / 2	46.5	50.8 / 47.9	90.0 / 82.5	$B \geq 80, B-A \geq 20$ pp, $p < 0.01$ in each seed	passed
S2	div3	50.7	[TBD]	[TBD]	≥ 2 of the 3 with $B-A \geq 15$ pp in both seeds	[TBD]
	div11	50.3	[TBD]	[TBD]		
	div13	53.3	49.6	54.6		
S3	div2	100.0	[TBD]	[TBD]	$A \geq 95$	[TBD]
S4	div7, 3B model		[TBD]	[TBD]	$B > A$	[TBD]
S5	div7→6-digit			[TBD]	$ B - p^6 \leq 10$ pp; $p^6 = 75.8$	[TBD]
S6	prime→hard set		[TBD]	[TBD]	$A < 60$ and agrees more with “odd, no factor ≤ 7 ” than with the label	[TBD]

Training signal or test-time computation (B’). In B, the model may simply run the procedure at test time. B’ puts the answer first, so the procedure cannot compute it; if B’ exceeds A, trace tokens change what the model learns, and if B’ matches A, the gain is test-time computation learned from demonstrations. B’ on div7: [TBD]%.

Prompting the base model. A zero-shot CoT prompt gives [TBD]%, [TBD]% and [TBD]% on prime, div7 and valid, and 4 trace demonstrations in the prompt give [TBD]%, [TBD]% and [TBD]%. These separate “the base model runs the procedure when shown it” from “fine-tuning on traces taught it”.

5 SCALE

The map locates the answer frontier by the probe, which does not depend on the model, and the trace frontier by p^k , which should move with model size. Under the same protocol, Qwen2.5-3B-Instruct gives, as A/B: prime [TBD]/[TBD], div7 [TBD]/[TBD], valid [TBD]/[TBD] (S4 [TBD]). Where compute allowed, div7 at 0.5B gives [TBD]/[TBD] and at 7B [TBD]/[TBD]. The map predicts that A stays near the probe at every size, while B’s accuracy on a fixed task rises with p ; a larger model should also narrow B’s deficit on primality only if its per-step accuracy approaches 1, since k reaches 25.

6 WHY: TRACES TURN DIVISIBILITY INTO A TABLE

Remark 1 (informal). Let $x_1 \cdots x_k$ be the decimal digits of n and $\delta_d(r, a) = (10r + a) \bmod d$. Then $d \mid n$ iff $\delta_d^*(0, x_1 \cdots x_k) = 0$, where δ_d^* applies δ_d digit by digit: divisibility by d is recognized by a d -state automaton. The arm-B trace writes the states $r_1, \dots, r_k$. Under teacher forcing, each supervised state $r_t = \delta_d(r_{t-1}, x_t)$ is a function of two tokens already in the context, so n training traces are kn samples of a single table with $10d$ entries, whatever k is (720 samples of 70 entries for $d = 7, k = 4, n = 180$). Answer-only supervision gives n samples of the k -fold composition, and for d coprime to 10 its value is nearly independent of every proper subset of the digits: for $d = 7$ and $k = 4$, fixing any three digits leaves $\Pr(7 \mid n)$ between 0.10 and 0.22, against a marginal of 0.14.

In the terms of Abbe et al. (2024), the answer-only target is as global as the input allows, while each trace step is local given the previous state; their inductive scratchpad, the teacher-forced CoT of Kim & Suzuki (2025) and the sub-task decomposition of Wies et al. (2023) are settings in which this kind of decomposition provably makes learning efficient. We do not claim the remark extends those results: they concern transformers trained from scratch on specified distributions, and we fine-tune a pretrained model on 180 examples. The remark says which of our tasks should benefit, and it makes three predictions.

(i) *Surface*. If d divides a power of 10 ($d = 2, 5$), the label is a function of the last digits, a probe finds it, and answers suffice (S3). (ii) *Table size*. For d coprime to 10 the label depends on every digit, so answers can succeed only through a rule over all digits (the digit sum for $d = 3$), and traces should help with a table of $10d$ entries, so at fixed n the benefit may shrink as d grows (S2). (iii) *Compounding*. If each state is right with probability p independently and a wrong state is uniform on $\mathbb{Z}_d$, the answer is right with probability $p^k + (1 - p^k)/d$ on multiples of d and $p^k + (1 - p^k)(d - 1)/d$ on non-multiples, hence $(1 + p^k)/2$ on a balanced test set (Appendix E). The preregistered S5 uses p^k , which is the prediction for the positive class up to $1/d$.

Primality through the same lens. A trial-division step asks the model to reduce a four-digit number modulo a prime in one step: a table indexed by (n, p) with more than 10^5 entries, almost none of which recur in training, rather than a $10d$ -entry table seen many times. A prime’s trace has 11–25 such steps, and the asymmetric per-class split of §4.2 is the signature (iii) predicts for a long trace in which one error flips a positive into a negative. A digit-level trace for each divisor would make every step local but multiply k by four; the map predicts that this trade fails unless p is very close to 1. We have not run it.

Validity. The trace is a certificate whose first clause (satisfiable, or contradiction) fixes the answer, so k is effectively 1, and the forms are shared with training; both arms work, as both conditions predict.

7 DISCUSSION AND LIMITATIONS

What exact labels guarantee. An exact verifier guarantees that no training label is wrong. It does not guarantee that the labels identify the verifier’s function among the functions a model can fit from n examples. On primality, exactly correct labels taught a last-digit rule that calls every composite ending in 1, 3, 7 or 9 prime, among them all 996 of the 7,939 4-digit composites without a factor ≤ 7 . A shuffled-label control shows that the gain comes from label correctness, but it cannot tell the verifier’s function from a surface rule; hard negatives chosen from the probe’s errors can.

Supervision design for verifiable-reward pipelines. Three practices follow from the map. (1) Fit a cheap surface probe on the verified data before training; where it is already accurate, answer supervision will teach its function, and the evaluation should include inputs on which the probe is wrong. (2) Where a trace is used, make each step a function of a few tokens already in context and keep k small, or measure the per-step accuracy and budget p^k ; a correct but long trace can be worse than no trace. (3) Do not read a null from one training size; our own 60-example null on div7 was the edge of a transition, not the absence of an effect.

No RLVR arm, and what the map predicts for one. With a binary verifier, the RLVR reward is a function of the final answer only, so the map treats RLVR as answer supervision with on-policy sampling. On a surface-learnable task, a surface rule already earns most of the reward on the training distribution (the last-digit rule A found is right on 82.5% of our prime test items, “no factor ≤ 7 ” on 93.8%), and the map predicts that RLVR finds a shortcut of the same kind. On div7 the reward carries no gradient toward the procedure unless the policy’s own samples sometimes execute it; the base model’s zero-shot 62.5% shows that its free-form outputs carry some signal, and Yue et al. (2025) suggest that RLVR mostly reweights such samples. We did not run GRPO or any RL method. It is the first experiment we would add, evaluated on the hard-negative set.

Limitations. One model family (Qwen2.5), with 1.5B as the main size; LoRA with one configuration. Binary tasks with balanced test sets of 240 items, so effects under about 6 pp are not resolved, and the per-class and hard-negative figures rest on 120 and 15 items. One trace format per task, written by us; the map predicts how other formats should behave, but we tested one. The div7 result rests on three seeds of one model size. The first preregistration’s conditional arm D (a fluent trace from another instance, close to the corrupted traces of Valmeekam et al. (2026)) now qualifies on div7 and has not been run. The shuffled-label control comes from an earlier study with a related protocol (answer-only prompts, 3–4-digit numbers). The probe is one choice, defined after the prior grid; a stronger probe would move the answer frontier. The logic task shares 12 forms between training

and test and tests surface learnability, not logical reasoning. Fine-tuning on answers also removes the base model’s test-time reasoning, a confound that B’ and the prompting controls address.

AI USE STATEMENT

We used an AI assistant (Claude, Anthropic) to write and refactor experiment code, to run the experiments and log their outputs under our direction, to compute the statistics, to draft and edit the manuscript, and to check every bibliography entry against arXiv, OpenReview, proceedings and DOI records. The research question, the arms, the preregistered decision rules and the interpretation were reviewed and approved by the authors before the runs they govern. We reviewed all AI-assisted code, text and numbers, verified reported numbers against the released result files, and take responsibility for the content of this paper.

REPRODUCIBILITY STATEMENT

The supplementary material contains the dataset generators (with fixed seeds), the single-run training and evaluation worker, the queue runner, both preregistration files and the commit that froze the second, and every run’s result record, including a per-item correctness bitmap from which each paired test in this paper can be recomputed. Section 3 and Appendix C give the prompts, trace formats and hyperparameters. All numbers in the text are macros regenerated from the result files by scripts in the `paper/` directory, including the surface probe and the derived per-class statistics (`paper/exp/make_numbers.py`, `make_figures.py`, `build_all.py`). Each run takes 1–17 minutes on one 8 GB consumer GPU.

ETHICS STATEMENT

The study uses synthetic arithmetic and logic data only and involves no human subjects or personal data.

REFERENCES

- Emmanuel Abbe, Samy Bengio, Aryo Lotfi, Colin Sandon, and Omid Saremi. How far can transformers reason? The globality barrier and inductive scratchpad. In *Advances in Neural Information Processing Systems*, 2024.
- François Charton. Learning the greatest common divisor: explaining transformer predictions. In *International Conference on Learning Representations*, 2024.
- Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Dale Schuurmans, Quoc V. Le, Sergey Levine, and Yi Ma. SFT memorizes, RL generalizes: A comparative study of foundation model post-training. In *International Conference on Machine Learning*, 2025.
- Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, Lecture Notes in Computer Science, pp. 337–340. Springer, 2008. doi: 10.1007/978-3-540-78800-3_24.
- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Peter West, Chandra Bhagavatula, Ronan Le Bras, Jena D. Hwang, Soumya Sanyal, Sean Welleck, Xiang Ren, Allyson Ettinger, Zaid Harchaoui, and Yejin Choi. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems*, 2023.
- Guhao Feng, Bohang Zhang, Yuntian Gu, Haotian Ye, Di He, and Liwei Wang. Towards revealing the mystery behind chain of thought: A theoretical perspective. In *Advances in Neural Information Processing Systems*, 2023.

- Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020. doi: 10.1038/s42256-020-00257-z.
- Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- Juno Kim and Taiji Suzuki. Transformers provably solve parity efficiently with chain of thought. In *International Conference on Learning Representations*, 2025.
- Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, et al. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.
- Nayoung Lee, Kartik Sreenivasan, Jason D. Lee, Kangwook Lee, and Dimitris Papailiopoulos. Teaching arithmetic to small transformers. In *International Conference on Learning Representations*, 2024.
- Zhiyuan Li, Hong Liu, Denny Zhou, and Tengyu Ma. Chain of thought empowers transformers to solve inherently serial problems. In *International Conference on Learning Representations*, 2024.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations*, 2024.
- Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.
- Terufumi Morishita, Gaku Morio, Atsuki Yamaguchi, and Yasuhiro Sogawa. Enhancing reasoning capabilities of LLMs via principled synthetic logic corpus. In *Advances in Neural Information Processing Systems*, 2024.
- Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*, 2021.
- Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- Amrith Setlur, Saurabh Garg, Xinyang Geng, Naman Garg, Virginia Smith, and Aviral Kumar. RL on incorrect synthetic data scales the efficiency of LLM math reasoning by eight-fold. In *Advances in Neural Information Processing Systems*, 2024.
- Shai Shalev-Shwartz, Ohad Shamir, and Shaked Shammah. Failures of gradient-based deep learning. In *International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pp. 3067–3075, 2017.
- Zayne Sprague, Fangcong Yin, Juan Diego Rodriguez, Dongwei Jiang, Manya Wadhwa, Prasann Singhal, Xinyu Zhao, Xi Ye, Kyle Mahowald, and Greg Durrett. To CoT or not to CoT? Chain-of-thought helps mainly on math and symbolic reasoning. In *International Conference on Learning Representations*, 2025.
- Karthik Valmeekam, Vardhan Palod, Kaya Stechly, Atharva Gundawar, and Subbarao Kambhampati. Beyond semantics: The unreasonable effectiveness of reasonless intermediate tokens. *Transactions on Machine Learning Research*, 2026.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.

Noam Wies, Yoav Levine, and Amnon Shashua. Sub-task decomposition enables learning in sequence to sequence tasks. In *International Conference on Learning Representations*, 2023.

Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems*, 2025.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems*, 2022.

A FULL RESULTS

Table 4 lists every run as logged. “B-ext” uses reasoning traces sampled from a locally served 9B model and kept only when their final answer matched the verifier (STaR-style rejection sampling); only 9 traces passed, so these runs are not comparable to B and are listed for completeness. Train tokens count whitespace tokens in the completions. Table 5 reports the earlier answer-only study with its shuffled-label control.

Table 4: Every run, as logged (240 test items per cell). Cell: the test set, prefixed by the training task when they differ; [cot] and [fewshot4] mark prompting controls on the base model. Train tokens: whitespace tokens in the completions.

model	cell	arm	n	seed	train tokens	accuracy (%)
1.5B	prime	base	0	0	0	59.6
1.5B	prime	A	60	0	120	59.6
1.5B	prime	A	180	0	360	82.1
1.5B	prime	A	180	1	360	82.5
1.5B	prime	A	540	0	1,080	75.0
1.5B	prime	B	60	0	4,355	57.1
1.5B	prime	B	180	0	13,370	57.9
1.5B	prime	B	180	1	13,260	57.5
1.5B	prime	B	540	0	40,095	68.8
1.5B	prime	C	180	0	12,561	50.0
1.5B	prime	C	180	1	13,120	54.2
1.5B	prime	A-full	1320	0	2,640	80.8
1.5B	prime	B-ext	9	0	896	62.1
1.5B	prime	B-ext	9	1	974	60.0
1.5B	div7	base	0	0	0	62.5
1.5B	div7	A	60	0	120	50.0
1.5B	div7	A	180	0	360	52.1
1.5B	div7	A	180	1	360	50.8
1.5B	div7	A	180	2	360	47.9
1.5B	div7	A	540	0	1,080	50.0
1.5B	div7	B	60	0	3,600	50.0
1.5B	div7	B	180	0	10,800	92.5
1.5B	div7	B	180	1	10,800	90.0
1.5B	div7	B	180	2	10,800	82.5
1.5B	div7	C	180	0	10,784	49.6
1.5B	div7	C	180	1	10,824	50.4
1.5B	valid	base	0	0	0	30.0
1.5B	valid	A	60	0	120	97.5
1.5B	valid	A	180	0	360	99.2
1.5B	valid	A	180	1	360	100.0
1.5B	valid	A	540	0	1,080	100.0
1.5B	valid	B	60	0	2,343	87.5
1.5B	valid	B	180	0	7,001	100.0
1.5B	valid	B	180	1	7,004	80.0
1.5B	valid	B	540	0	21,084	100.0
1.5B	valid	C	180	0	7,092	54.2
1.5B	valid	C	180	1	7,090	50.8

model	cell	arm	n	seed	train tokens	accuracy (%)
1.5B	valid	A-full	1600	0	3,200	100.0
1.5B	div13	A	180	0	360	49.6
1.5B	div13	B	180	0	10,800	54.6

Table 5: Earlier answer-only study (same model and LoRA configuration; 240 training items, answer-only prompts, 3–4-digit numbers for arithmetic; mean of two seeds). Correct labels vs. labels permuted across training items.

task	base	correct labels	shuffled labels
divisible by 3	55.0	67.3	49.6
divisible by 7	53.3	51.0	52.5
divisible by 13	54.2	50.0	50.0
perfect square	51.2	83.3	51.5
prime	50.0	82.9	50.0
argument validity	66.7	99.4	50.0
joint consistency (SAT)	50.0	99.8	48.5

B THE LOGIC TASK

Arguments instantiate 6 valid forms (modus ponens, modus tollens, hypothetical syllogism, disjunctive syllogism, constructive dilemma, contraposition) and 6 invalid ones (affirming the consequent, denying the antecedent, affirming a disjunct, a broken chain, the converse, and “or” to “and”) with distinct random atom letters, rendered in English (“if A then B ”, “not A ”, “ A or B ”). The schema proposes an argument and Z3 decides it: the argument is labeled valid iff the premises together with the negated conclusion are unsatisfiable. For a valid argument the trace names the unsatisfiable core; for an invalid one it gives the satisfying assignment as a countermodel. Training and test share the 12 forms and never an instance, so a model can reach high accuracy by recognizing the form, which the atom-renamed n -gram probe does (100.0%). The task therefore checks that answers work when the surface determines the label; it is not a test of logical reasoning. Its results: base 30.0%, A 99.6% (99.2 and 100.0), B 90.0% (100.0 and 80.0), C 52.5% (54.2 and 50.8); at $n = 60$, A 97.5% and B 87.5%; at $n = 540$, both 100.0%.

C TRACE FORMATS

All arms share the prompt, which ends “End your reply with ‘Answer: Yes’ or ‘Answer: No.’” Examples of training completions, verbatim:

div7, B (prompt: “Is 1617 divisible by 7?”):

```
Process the digits of 1617 keeping a running remainder mod
7.  r = (10*0 + 1) mod 7 = 1.  r = (10*1 + 6) mod 7 = 2.  r
= (10*2 + 1) mod 7 = 0.  r = (10*0 + 7) mod 7 = 0.  Final
remainder 0, so 1617 is divisible by 7.
Answer: Yes
```

div7, C (a scrambled trace from another example; the answer line keeps this example’s label):

```
running mod r = so = 4067 Process by 7.  7 a = 0.  0) 0.  7
mod (10*5 = r 7 the = = keeping + r is mod mod 4) (10*0 + of
4067 r + 7) 5.  = + digits remainder 7.  (10*0 = 7 0,
divisible Final 6) 4.  (10*4 mod remainder
Answer: Yes
```

prime, B, a composite (“Is 4143 a prime number?”):

Check prime divisors up to $\text{isqrt}(4143) = 64$. $4143 \bmod 2 = 1$. $4143 \bmod 3 = 0$. 3 divides 4143, so 4143 is composite.
Answer: No

prime, B, a prime (“Is 1493 a prime number?”):

Check prime divisors up to $\text{isqrt}(1493) = 38$. $1493 \bmod 2 = 1$. $1493 \bmod 3 = 2$. $1493 \bmod 5 = 3$. $1493 \bmod 7 = 2$. $1493 \bmod 11 = 8$. $1493 \bmod 13 = 11$. $1493 \bmod 17 = 14$. $1493 \bmod 19 = 11$. $1493 \bmod 23 = 21$. $1493 \bmod 29 = 14$. $1493 \bmod 31 = 5$. $1493 \bmod 37 = 13$. No prime divisor up to 38, so 1493 is prime.
Answer: Yes

valid, B, invalid (“Premises: (1) if H then D; (2) D. Conclusion: H.”):

Assume every premise is true and the conclusion false. That is satisfiable: set $D=\text{true}$, $H=\text{false}$. Under this assignment all premises hold while the conclusion is false, a countermodel, so the argument is invalid.
Answer: No

valid, B, valid (“Premises: (1) if S then R; (2) S. Conclusion: R.”):

Assume every premise is true and the conclusion false, i.e. $\sim(R)$ holds. Together these force a contradiction: $\{S \rightarrow R, S, \sim(R)\}$ cannot all hold at once. Since assuming the conclusion false is impossible, the conclusion follows.
Answer: Yes

A (every task): Answer: Yes or Answer: No.

D PREREGISTRATION TEXT

First preregistration (prior grid). Frozen before any run of the prior grid and never edited; results were appended below it. It fixed: arms A, B, C, an answers-only arm at larger scale, arms using model-generated traces, and a conditional wrong-trace arm; Qwen2.5-1.5B-Instruct with the LoRA configuration of §3; tasks prime, valid and div7, all reported; 240 problem-disjoint test items per task (120 per class), identical across arms, graded by one parser on the final answer. Endpoints: H1, $B > A$ at $n = 180$ per task, paired McNemar, Holm across the three tasks; H2, $B > C$ at $n = 180$; H3, accuracy at matched training tokens; H4, on div7 B reaches about 60% with A at chance. The full text is in the supplementary material.

Second preregistration (this paper’s new runs). Reproduced from PREDICTIONS.md; Mark-down markup converted, text unchanged except one redacted location.

Preregistered predictions and decision rules (frozen before any run listed here). Committed 2026-09-25 ~20:35 [location redacted], before any of the runs below were started. The commit hash of this file is the preregistration. Earlier results (results/THINKING_VS_DATA.md, frozen 2026-08-04, and its later staged results block) are prior data, reported as such.

Provenance of the prior div7 result (stated in the paper). The 2026-08-04 verdict “H4 (a procedure trace unlocks div7): NO” was written from the $n = 60$ trace run alone ($B@60 = 50.0\%$). The preregistered main cell, $B@180$ seed 0, finished afterwards at 92.5% with the same worker (no code change). The paper reports the premature verdict and the later result.

Claim under test. Exact labels teach shortcuts; exact traces teach short programs. With labels from an exact verifier: answer-only supervision (arm A) recovers roughly what a simple probe on the answer surface recovers; trace supervision (arm B: procedure trace, then answer) succeeds when the per-step trace accuracy p , compounded over k steps, stays high (predicted accuracy about p^k), and fails when the procedure is long.

Decision rules. S1 (go/no-go, before any other new run): div7 $B@180$ seeds 1 and 2 each reach $\geq 80\%$ accuracy, with $B - A \geq 20$ percentage points and McNemar $p < 0.01$

against A of the same seed. If S1 fails, the paper is not submitted in this form. **S2:** at least 2 of {div3, div11, div13} (4-digit numbers) give $B - A \geq 15$ pp with the same sign in both seeds. **S3:** div2 $A \geq 95\%$ (the answer is surface-learnable from the last digit). **S4:** the div7 sign ($B > A$) holds for Qwen2.5-3B-Instruct. **S5 (falsifiable length test):** on 6-digit div7, B's accuracy is within 10 pp of p^6 , where p is B's per-step accuracy measured on 4-digit div7 generations. **S6 (shortcut):** A on prime agrees more with the rule "odd and no factor ≤ 7 " than with the true label on hard negatives (odd composites with no factor ≤ 7), where A's accuracy drops below 60%.

Definitions: a "win" is a difference of ≥ 10 pp with both seeds agreeing in sign; "neither" is both arms at or below majority class + 8 pp. McNemar tests per seed, Holm correction across tasks within each hypothesis family, $\alpha = 0.05$. Every cell is reported, including those that go against the claim.

Arms. base (zero-shot), A (answers), B (trace then answer), C (length-matched scrambled trace), B' (answer then trace, div7 only), plus eval-only controls: zero-shot chain-of-thought prompt and 4-shot trace demonstrations on the base model. Model: Qwen2.5-1.5B-Instruct (laptop), Qwen2.5-3B-Instruct and others where the RTX 5090 allows. LoRA $r = 8$, $\alpha = 16$, 3 epochs, $lr = 2e-4$, bf16, greedy decoding, $n = 180$ training examples, 240 problem-disjoint test items.

E THE COMPOUNDING APPROXIMATION

Let an item have true final state r_k^* and suppose each emitted state is correct with probability p , independently, and that after the first error the emitted final state is uniform on $\mathbb{Z}_d$. With probability p^k the trace is error-free and the answer is right. Otherwise the answer is right if the emitted final state falls on the correct side of the test $r_k = 0$: with probability $1/d$ when $r_k^* = 0$ (a multiple of d) and $(d-1)/d$ otherwise. On a balanced test set the accuracy is therefore $p^k + \frac{1}{2}(1-p^k)[\frac{1}{d} + \frac{d-1}{d}] = (1+p^k)/2$. Two assumptions can fail: errors can be correlated across steps (a model that miscomputes one table entry does so whenever the entry recurs), and a wrong state need not be uniform. The positive-class accuracy, $p^k + (1-p^k)/d$, is the cleaner test of compounding, because a model that answers "no" by default also scores well on non-multiples: on 6-digit div7 it is **[TBD]**%.